

WEEK - 08

Implement Johnson Trotter algorithm to generate permutations.

```
Start here X *john.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define LEFT -1
5  #define RIGHT 1
6
7  // Function to print a permutation
8  void printPermutation(int perm[], int n) {
9      for (int i = 0; i < n; i++)
10         printf("%d ", perm[i]);
11     printf("\n");
12 }
13
14 // Function to find the largest mobile element
15 int getMobile(int perm[], int dir[], int n) {
16     int mobile = 0;
17     int mobile_index = -1;
18
19     for (int i = 0; i < n; i++) {
20         if (dir[i] == LEFT && i != 0 && perm[i] > perm[i - 1]) {
21             if (perm[i] > mobile) {
22                 mobile = perm[i];
23                 mobile_index = i;
24             }
25         }
26         if (dir[i] == RIGHT && i != n - 1 && perm[i] > perm[i + 1]) {
27             if (perm[i] > mobile) {
28                 mobile = perm[i];
29                 mobile_index = i;
30             }
31         }
32     }
33     return mobile_index;
34 }
35
36 // Johnson-Trotter Algorithm
37 void johnsonTrotter(int n) {
38     int *perm = (int *)malloc(n * sizeof(int));
39     int *dir = (int *)malloc(n * sizeof(int));
40
41     // Initialize permutation and directions
42     for (int i = 0; i < n; i++) {
43         perm[i] = i + 1;
44         dir[i] = LEFT;
45     }
46     printPermutation(perm, n);
```

```

47 while (1) {
48     int mobile_index = getMobile(perm, dir, n);
49
50     if (mobile_index == -1)
51         break;
52
53     int swap_index = mobile_index + dir[mobile_index];
54
55     // Swap elements
56     int temp = perm[mobile_index];
57     perm[mobile_index] = perm[swap_index];
58     perm[swap_index] = temp;
59
60     // Swap directions
61     int tempDir = dir[mobile_index];
62     dir[mobile_index] = dir[swap_index];
63     dir[swap_index] = tempDir;
64
65     int mobile = perm[swap_index];
66
67     // Reverse direction of elements greater than mobile
68     for (int i = 0; i < n; i++) {
69         if (perm[i] > mobile)
70             dir[i] = -dir[i];
71     }
72
73     printPermutation(perm, n);
74 }
75
76 free(perm);
77 free(dir);
78 }
79
80 int main() {
81     int n;
82
83     printf("Enter number of elements: ");
84     scanf("%d", &n);
85
86     johnsonTrotter(n);
87
88     return 0;
89 }
90

```

```

"C:\Users\BMSCECSE\Documents"
Enter number of elements: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 1.700 s
Press any key to continue.

```

```
"C:\Users\BMSCECSE\Docume  X + v
Enter number of elements: 4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

Process returned 0 (0x0)   execution time : 2.057 s
Press any key to continue.
```